

CPSC 250

Basic Plotting of Data

Edward Brash

1 Motivation

We have already seen examples where plotting data is useful.

In this lecture, we develop a base plotting example using `matplotlib`.

The goal is to create a plot with:

1. a title,
2. axis labels,
3. data points with error bars,
4. a fit line,
5. a legend,
6. appropriate axis limits,
7. and possibly logarithmic axes or an error band.

2 Input Data Format

Suppose we have data stored in a CSV file.

The file has columns:

$$x, \quad y, \quad \Delta x, \quad \Delta y$$

where:

- x is the independent variable,
- y is the dependent variable,
- Δx is the uncertainty in x ,
- Δy is the uncertainty in y .

A schematic CSV file might look like:

```
XLabel , YLabel , dXLabel , dYLabel
1.0 , 3.0 , 0.01 , 0.03
2.0 , 3.2 , 0.02 , 0.02
...
```

3 What We Want to Plot

A good data plot should show:

- the data points,
- both x and y error bars,
- the best-fit curve,
- a legend describing the data and fit,
- the x-axis label,
- the y-axis label,
- a title.

4 Reading the Data

We can use Python's `csv` module to read the data.

```
import csv

def read_data(filename):

    x = []
    y = []
    dx = []
    dy = []

    with open(filename, "r") as file:

        reader = csv.reader(file)

        headers = next(reader, None)

        for row in reader:

            x.append(float(row[0]))
            y.append(float(row[1]))
            dx.append(float(row[2]))
            dy.append(float(row[3]))

    return headers, x, y, dx, dy
```

5 The Header Row

The line:

```
headers = next(reader, None)
```

reads the first row of the file.

This row contains labels rather than data.

For example:

```
XLabel, YLabel, dXLabel, dYLabel
```

We can use these labels later for axis labels.

6 Creating the Basic Plot

We import:

```
import matplotlib.pyplot as plt
```

Then:

```
filename = "testdata.csv"

headers, xi, yi, dxi, dyi = read_data(filename)

plt.errorbar(xi, yi, xerr=dxi, yerr=dyi, fmt="o")

plt.title("Basic Plotting Example")
plt.xlabel(headers[0])
plt.ylabel(headers[1])

plt.show()
```

7 The errorbar() Function

The function:

```
plt.errorbar()
```

plots points with error bars.

In this example:

```
plt.errorbar(xi, yi, xerr=dxi, yerr=dyi, fmt="o")
```

means:

- use `xi` as x-values,
- use `yi` as y-values,
- use `dxi` as x uncertainties,
- use `dyi` as y uncertainties,
- use circular markers.

8 Axis Limits

Matplotlib will choose default axis limits automatically.

However, sometimes we should set the limits ourselves.

For example:

```
plt.ylim(0, 0)
```

is a special case: Python interprets this as a request to choose the upper limit automatically while forcing the lower limit to zero.

This can be useful when the physical quantity cannot be negative.

9 Which Plot Requirements Are Already Satisfied?

At this stage, we have:

- a title,
- x-axis label,
- y-axis label,
- data points with x and y error bars,
- appropriate automatic axis behavior.

We have not yet added:

- a fit curve,
- a legend,
- logarithmic axes,
- an error band.

10 Fitting

Fitting is a very large topic.

In this course, we use a simple practical approach.

The steps are:

1. Choose a fitting function.
2. Call a fitting routine.
3. Extract the best-fit parameters.
4. Plot the fit.

11 Choosing a Fit Function

Suppose the data appear roughly quadratic.

Then we might choose:

$$y = ax^2 + bx + c$$

In Python:

```
def fitfunction(x, *param):  
    return param[0]*x*x + param[1]*x + param[2]
```

The star in:

```
*param
```

allows the function to accept a variable number of parameters.

Here:

- `param[0]` is a ,
- `param[1]` is b ,
- `param[2]` is c .

12 Calling `curve_fit`

We import:

```
from scipy.optimize import curve_fit
```

Then we call:

```
init_vals = [0.0 for x in range(3)]

popt, pcov = curve_fit(
    fitfunction,
    xi,
    yi,
    p0=init_vals,
    sigma=dyi,
    absolute_sigma=True
)
```

Here:

- `popt` contains the optimal fit parameters,
- `pcov` contains the covariance matrix.

13 Parameter Uncertainties

The uncertainties in the fitted parameters come from the diagonal of the covariance matrix:

```
import numpy as np

perr = np.sqrt(np.diag(pcov))
```

Then:

```
a = pop_t[0]
b = pop_t[1]
c = pop_t[2]

da = perr[0]
db = perr[1]
dc = perr[2]
```

This gives:

$$a \pm \Delta a$$

$$b \pm \Delta b$$

$$c \pm \Delta c$$

14 Plotting the Fit

To plot a smooth curve, create many x-values:

```
xlow = min(xi) - 2
xhigh = max(xi) + 2

xfit = np.linspace(xlow, xhigh, 100)
yfit = fitfunction(xfit, *popt)

plt.plot(xfit, yfit, "--")
```

The line:

```
fitfunction(xfit, *popt)
```

evaluates the fit function using the fitted parameters.

15 Key Takeaways

- CSV files are useful for storing data.
- A good plot includes labels, a title, and a legend.
- Error bars communicate measurement uncertainty.
- `plt.errorbar()` plots data with uncertainties.
- `curve_fit()` performs numerical fitting.
- The covariance matrix provides parameter uncertainties.
- Smooth fit curves are created using many x-values.